Ricorsione

Materia	Algoritmi
Anno	Secondo Anno
Data	@October 8, 2024
Studiato	Done

L'indicizzazione degli array parte da 1 e arriva ad n, quindi $A[1 \dots n]$

Cominciamo parlando della funzione **fattoriale** definita nel seguente modo:

$$\begin{cases} f(1) = 1 \\ f(n) = n \cdot f(n-1) \end{cases}$$

Diremo che 1 è il **caso base**, mentre n è il **caso ricorsivo**.

Descrivere il fattoriale nel seguente modo è simile a dimostrare **per induzione** che è possibile calcolare $f(n)$, perché $f(1) = 1$, e a partire da $f(n-1)$ possiamo calcolare $f(n)$ come illustrato.

Allora la nostra funzione ricorsiva sarà la seguente:

```
fattoriale(n):
    if n = 1 then return 1
    return fattoriale(n-1) * n
```

Abbiamo anche una versione iterativa:

```
fattoriale(n):
    p = 1
    for i = 2 to n:
        p = p * i
    return p
```

Possiamo anche vedere come l'operazione di moltiplicazione nel seguente modo:

```
mul(x,y):
    if y = 1 then return x
    return x + mul(x, y-1)
```

Un altro problema noto è la **sequenza di Fibonacci:**

$$1, 1, 2, 3, 5, 8, 13, 21...$$

$$\begin{cases} f(1) = 1 \\ f(2) = 1 \\ f(n) = f(n-1) + f(n-2) \end{cases}$$

In maniera ricorsiva viene implementato nel seguente modo:

```
fib(n):
    if n <= 2 then return 1
    return fib(n-1) + fib(n-2)
```

Tuttavia il problema di questo algoritmo è che ha **complessità esponenziale**. Nell'albero di ricorsione avremo che ogni nodo o non ha figli (quindi le foglie sono **i casi base**) oppure ne ha esattamente due. L'algoritmo quindi **risolve più volte lo stesso sotto-problema**, solo che **non tiene traccia che lo abbia realmente risolto**.

Nella situazione descritta precedentemente la ricorsione pura non è realmente utile. Quindi è necessario **utilizzare la memorizzazione delle soluzione**, ossia allochiamo un array F di n locazioni, dove n è la dimensione del problema. La cella $F[i]$ conterrà la soluzione del problema nella istanza i. Nel caso di Fibonacci, $F[i]$ contiene l'i-esimo elemento della sequenza di Fibonacci. In particolare sappiamo che $F[1] = F[2] = i$.

Allora la nostra funzione diventa:

```
fib(n):
    if n <= 2 then return F[n]
    if F[n-1] = null then
```

```
    F[n-1] = fib(n-1)
  return F[n-1] + F[n-2]
```

Notiamo che nel momento in cui stiamo calcolando $F[n-1]$, abbiamo obbligatoriamente bisogno di $F[n-2]$, quindi un secondo `if` per il controllo `F[n-2] = null` non è realmente necessario. Di conseguenza, nell'albero della ricorsione, molti casi verranno considerati come **casi base** e l'algoritmo diventa quasi lineare.

Diciamo quindi che l'approccio **iterativo** è un approccio **bottom-up**, perché da un problema più piccolo costruiamo la soluzione del problema più grande, mentre l'approccio ricorsivo è **top-down**, ossia affrontiamo i problemi più grandi che richiedono i problemi più piccoli. Inoltre, un approccio bottom-up andrebbe a risolvere tutti i sotto-problemi, mentre un approccio top-down **andrebbe a considerare solo i sotto-problemi necessari presenti in un insieme di sotto-problemi di** P. Quindi per evitare di risolvere lo stesso problema, utilizziamo la memorizzazione.

Problema dello zaino

Il **problema dello zaino** è un problema noto presente in diverse formulazioni.

Abbiamo uno zaino che ha capienza massima in termini di peso pari a k. Inserendo un numero di oggetti tale che il peso è maggiore di k, lo zaino si rompe. Abbiamo un insieme A di n oggetti che possiamo rubare. La somma dei pesi non può essere mai superiore a k ovviamente. Sia $S \subseteq A$ l'insieme di oggetti che portiamo. Allora il nostro limite è:

$$\sum_{x \in S} p(x) \leq k$$

dove $p(x)$ è una funzione che associa ad ogni elemento x il suo peso. La domanda che ci poniamo è **trovare il sottoinsieme** S **tale che** $|S|$ **è massima**.

Sia Z la funzione che definisce il problema. Z dipende dalla capienza dello zaino k e da n oggetti. Allora diremo $Z(k, n)$ è la funzione del problema, supponendo A sia globale. Supponiamo, inoltre, che A sia **ordinato in funzione del peso di ogni oggetto e in ordine decrescente**.

Possiamo sicuramente dire che se $n = 0$, allora $Z(k, n) = 0$. Inoltre, se neanche l'n-esimo oggetto può entrare, allora $Z(k, n) = 0$. Nel caso generico, invece, avremo $Z(k, n) = 1 + Z(k - p(n), n - 1)$. Quindi la nostra funzione è:

$$Z(k,n) = \begin{cases} 0 & se\ n = 0 \\ 0 & se\ k < p(n) \\ 1 + Z(k - p(n), n - 1) & altrimenti \end{cases}$$

Questa funzione è lineare perché si ha solo 1 chiamata ricorsiva.

```
knapsack(k, n):
   if n = 0 then return 0
   if p(n) > k then return 0
   return 1 + knapsack(k-p(n), n-1)
```

Seconda variante

Complichiamo il problema. Supponiamo che gli oggetti che rubiamo sono di oro e di peso diverso. Il guadagno stavolta è in base al peso. Quindi stavolta cerchiamo la soluzione tale che:

$$\sum_{x \in S} p(x) = massima$$

Stavolta quindi il problema consiste nel trovare la soluzione con peso massimo, che non implica che **il numero di oggetti sia il più alto possibile**. Ragionando in maniera ricorsiva, supponiamo di avere la soluzione per $n - 1$ elementi. Consideriamo l'n-esimo elemento. Allora abbiamo due scelte: prendere o non prendere l'elemento. Se lo porto, allora il mio peso sarà dato da $Z(k - p(n), n - 1) + p(n)$, altrimenti sarà $Z(k, n - 1)$. **Sceglieremo il più grande tra i due**. Per il caso base, come nel caso precedente, se $n = 0$ restituiamo 0, altrimenti se $k < 0$ restituiremo $-\infty$, perché **scegliendo l'elemento più grande tra i due in ogni passo la soluzione con peso eccessivo sarà invalida**.

La nostra funzione sarà:

$$Z(k,n) = \begin{cases} 0 & se\ n = 0 \\ -\infty & se\ k < 0 \\ max(p(n) + Z(k - p(n), n-1), Z(k, n-1)) & altrimenti \end{cases}$$

Anche in questo caso A è un **array ordinato in senso decrescente**.

```
knapsack2(n, k):
   if k < 0 then return -inf
   if n = 0 then return 0
   return max(knapsack2(n-1, k), p(n) + knapsack2(n-1, k-p(n))
```

Terza variante

Complichiamo ulteriormente il problema aggiungendo, oltre al peso, anche il valore. In particolare, devono essere verificate due condizioni:

$$\sum_{x \in S} p(x) \leq k$$
$$\sum_{i \in S} v(x) = massima$$

Dove p è la **funzione di peso** e v è la **funzione di valore**. Quindi vogliamo un peso minore del limite imposto (k) **e il massimo valore**.

In realtà la funzione è la stessa della precedente, ma stavolta invece di $p(n)$ consideriamo $v(n)$:

$$Z(k,n) = \begin{cases} 0 & se\ n = 0 \\ -\infty & se\ k < 0 \\ max(v(n) + Z(k - p(n), n-1), Z(k, n-1)) & altrimenti \end{cases}$$